Final Project Report

RAG-Based Healthcare Query Assistant

A Multi-Agent Conversational System for Structured (SQL) and Unstructured (RAG) Hospital Data Retrieval

Submitted By	Aman Kumar
Role	Data Science Intern, Celebal Technologies
Institution	Maharishi Markandeshwar (Deemed to be University)

Table of Contents

1. Project Overview & Objective

The RAG-Based Healthcare Query Assistant is an AI-powered, multi-agent application developed as part of the Celebal Excellence Intern Program at Celebal Technologies. The system enables hospital staff to query both structured patient records and unstructured hospital policy documents through a single, natural-language conversational interface, removing the need to separately navigate a database console and a document repository.

At the core of the system is an Orchestrator Agent that inspects each incoming query, determines the user's intent, and routes the request to the appropriate specialised sub-agent — either a SQL-based agent for structured patient data or a Retrieval-Augmented Generation (RAG) agent for policy-related questions. This design allows the assistant to behave as a single point of contact while internally delegating work to the agent best suited to answer it.

2. Technology Stack

The application was built using a modern, cost-conscious AI engineering stack, summarised below.

Component	Technology Used
Language	Python 3.10+
LLM Provider	Groq API (llama-3.3-70b-versatile) — chosen for ultra-low latency and to bypass SQL-blocking firewalls
Agent Framework	LangChain — for agent orchestration, tool calling, and prompt templating
Vector Database	FAISS (Facebook AI Similarity Search)
Embeddings	HuggingFace sentence-transformers/all-MiniLM-L6-v2 (local, zero-cost)
Relational Database	SQLite, with indexed columns for query performance
Frontend / UI	Streamlit — dark-themed, interactive dashboard
Data Manipulation	Pandas, NumPy

3. Architecture & Multi-Agent System

The application is built around a three-agent architecture, each with a clearly scoped responsibility:

Orchestrator Agent (Router)

Analyses the user's prompt and classifies its intent into one of three categories: SQL (patient data), RAG (hospital policies), or UNKNOWN (out-of-domain). It also handles short, stateless follow-up commands, such as “list 20”, by recognising them as continuations of the prior query rather than new, unrelated requests.

NLP-to-SQL Agent

Translates natural-language questions into syntactically correct SQL queries, executes them against the SQLite database, and formats the raw returned rows into conversational, readable Markdown tables for the end user.

RAG Agent

Retrieves the top-k most relevant text chunks from the FAISS vector store in response to a policy-related question, and uses the retrieved context to generate a grounded, hallucination-resistant answer via the LLM.

4. Step-by-Step Implementation & Deliverables

Development proceeded in four distinct phases, each documented via a dedicated Jupyter notebook, before being finalised into a single Streamlit application.

Phase 1: Data Preparation — 01_data_preparation.ipynb

- Loaded a raw, messy 10,000-row synthetic patient CSV dataset.
- **Data Cleaning:** corrected highly inconsistent name casing (e.g. converting “daVId bEcKeR” to “David Becker”), standardised column names for SQL compatibility, handled missing values, and converted date strings to proper datetime objects.
- **Database Creation:** persisted the cleaned data into a SQLite database (healthcare.db) and created indexes on frequently queried columns — Blood_Type, Medical_Condition, Admission_Date, and Insurance_Provider — to improve query performance.

Phase 2: RAG Pipeline & Knowledge Base — 02_rag_pipeline.ipynb

- Generated five synthetic hospital policy documents covering Admission, Billing, Discharge, Insurance, and Emergency procedures.
- Chunked the documents using LangChain's RecursiveCharacterTextSplitter.
- Generated embeddings locally using a HuggingFace sentence-transformer model to avoid API costs.

- Stored the resulting embeddings in a FAISS vector database (faiss_index/).

Phase 3: Agent Testing — 03_agent_testing.ipynb

- Evaluated the Orchestrator's routing accuracy across SQL, RAG, and UNKNOWN query types.
- Tested the SQL Agent's robustness against messy, inconsistently-cased data using SQL LOWER() functions.
- Measured overall routing accuracy and average end-to-end response latency.

Phase 4: Integration & UI — app.py

- Built a professional, dark-themed Streamlit dashboard as the single user-facing interface.
- Added a sidebar with live system status indicators, a clear-chat control, and tech-stack information.
- Added dashboard metrics for Total Patients, Active Policies, and Response Time.
- Implemented backend logging (agent_routing.log) to record every user query and its routing decision, for auditability.

5. Key Engineering Challenges & Solutions

Building a production-style multi-agent system surfaced several real-world AI engineering problems. Each is documented below along with the diagnosis and the implemented fix.

Challenge 1: LLM Hallucinations — Inventing Non-Existent Columns

Problem: The SQL Agent occasionally invented column names that did not exist in the schema (e.g. patient_name instead of the actual column Name), which caused SQL execution errors.

Solution: The system prompt was updated to explicitly enumerate the exact allowed column names and to strictly forbid the LLM from inventing new ones.

Challenge 2: API Crashes on Empty Results (400 Bad Request)

Problem: When a SQL query returned zero rows, LangChain passed an empty string back to the LLM as a tool result. The Groq/Sarvam APIs strictly reject empty tool messages, which caused a 400 error and crashed the app.

Solution: A custom safe_output tool wrapper was created to intercept all tool executions. Whenever a result is None, an empty string, or an empty list, it safely returns the message “No results found.” instead of an empty payload.

Challenge 3: Context Window Overflow on Large Result Sets

Problem: Broad queries such as "all patients" returned thousands of rows, overflowing the LLM's context window and crashing the application.

Solution: The safe_output wrapper was extended to detect when a result string exceeds 4,000 characters. In that case it truncates the text and injects a [SYSTEM NOTE] instructing the model to summarise the data and ask the user to refine their query. The prompt was also updated to distinguish "list all" requests from requests scoped to a specific condition.

Challenge 4: Azure Firewall Blocking Legitimate SQL Payloads (403 Forbidden)

Problem: While using the Sarvam AI API, the Azure Application Gateway mistakenly flagged SQL keywords present in the request payload as a SQL injection attack and blocked the request.

Solution: The LLM backend was migrated to the Groq API, which avoided the firewall issue entirely and, as a side benefit, reduced end-to-end response latency to under 2 seconds.

Challenge 5: Stateless Handling of Follow-Up Queries

Problem: A follow-up such as "List 20", sent after an initial query like "Show female patients over 50", was misclassified by the Router as an out-of-domain query because it lacked explicit context.

Solution: The Orchestrator prompt was updated to explicitly recognise short, numerical follow-up commands as continuations of the previous SQL intent, preserving conversational context.

6. Final Project Deliverables

The completed project consists of the following artefacts:

Deliverable	Description
`app.py`	The final, working Streamlit multi-agent application.
`healthcare.db`	The cleaned, indexed synthetic patient database.
`faiss_index/ & policies/`	The hospital policy knowledge base and its vector store.
`notebooks/01_data_preparation.ipynb`	Data cleaning and database setup documentation.
`notebooks/02_rag_pipeline.ipynb`	RAG pipeline and vector store construction documentation.
`notebooks/03_agent_testing.ipynb`	Agent testing, latency, and routing-accuracy evaluation.
`agent_routing.log`	Backend log providing proof of routing decisions.

7. Screenshot

Figure 1: Healthcare Multi-Agent Assistant - Main dashboard interface

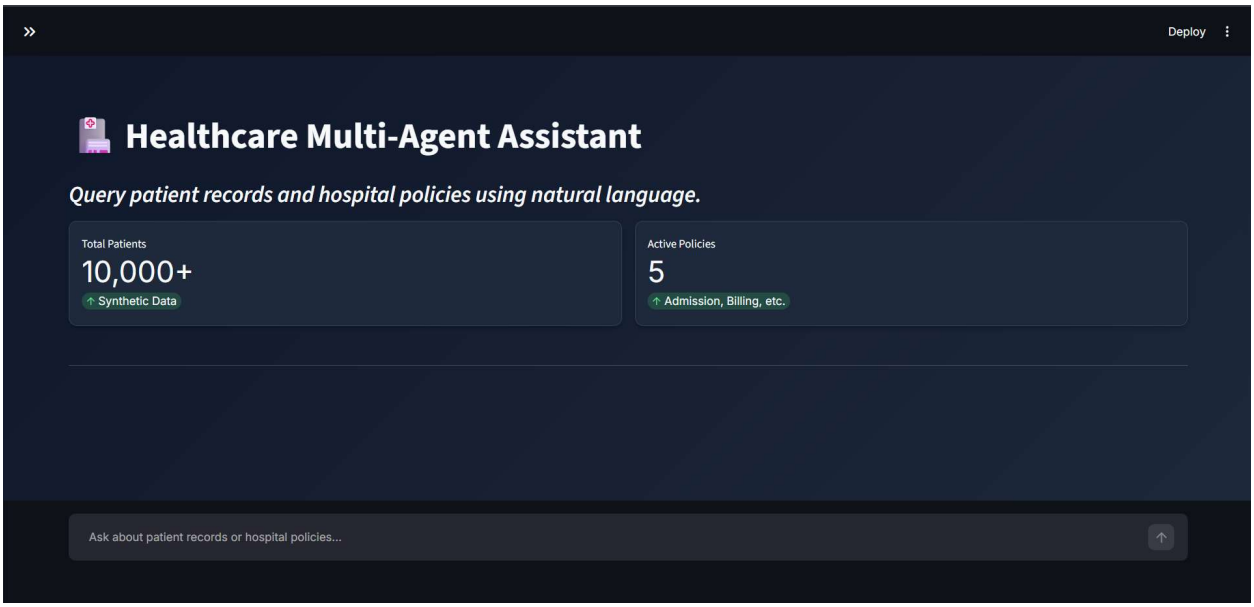


Figure 02: NLP-to-SQL Agent performing SQL aggregation to calculate average billing amounts for specific medical conditions

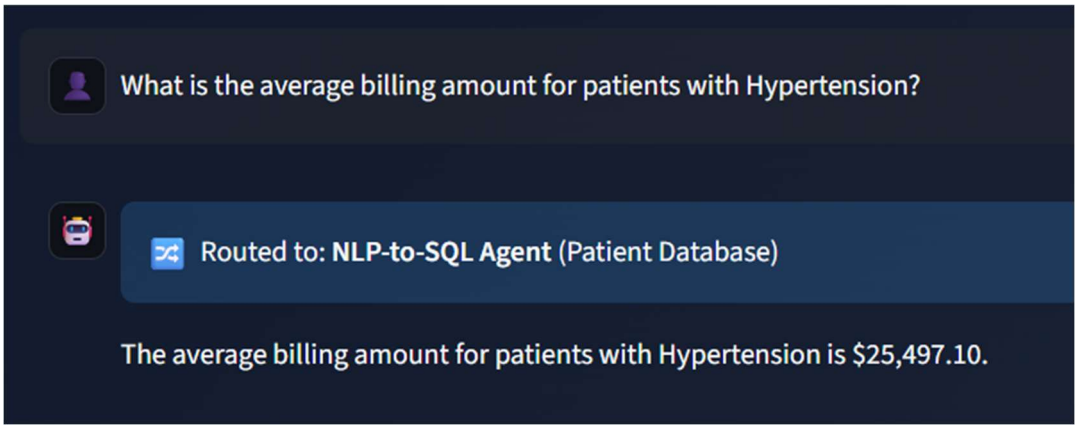


Figure 03: RAG Agent successfully retrieving and synthesizing hospital policy information from vectorized documents

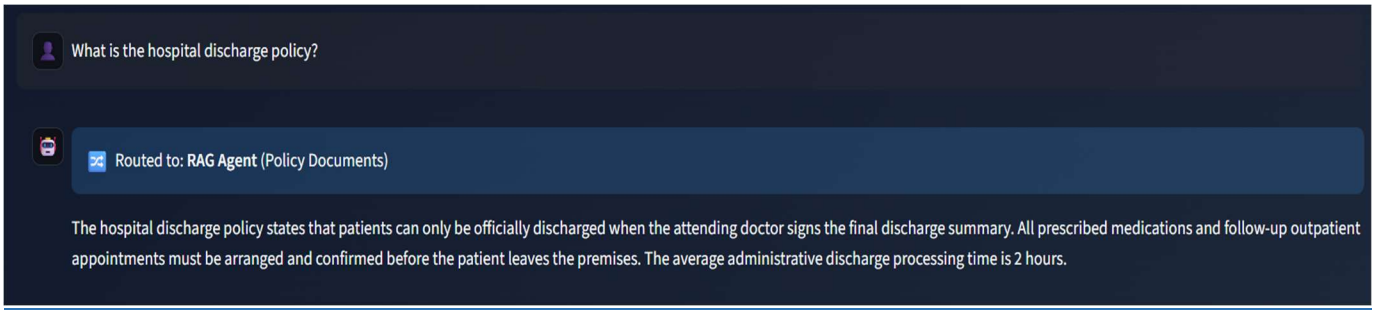
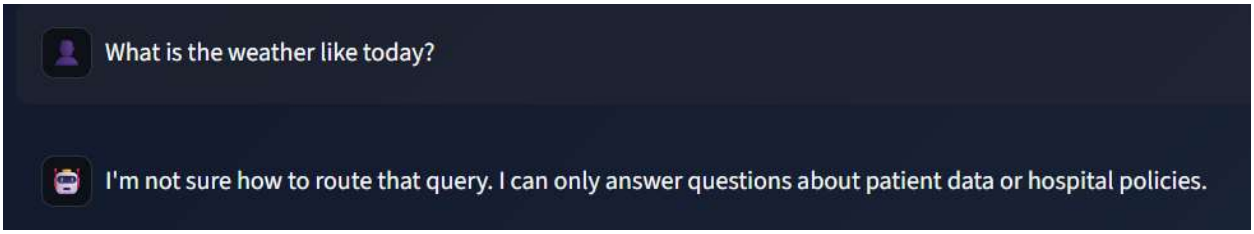


Figure04: Out-of-Domain Query Handling - Orchestrator Agent correctly identifying and rejecting non-healthcare queries (weather inquiry) with appropriate fallback response, demonstrating robust input validation and domain-specific constraint enforcement.



8. Conclusion

The RAG-Based Healthcare Query Assistant demonstrates an end-to-end, production-minded approach to building multi-agent LLM applications: a clear separation of concerns between structured and unstructured data retrieval, defensive engineering against common LLM failure modes (hallucinated schema references, empty-result crashes, and context overflow), and a polished, user-facing interface. The engineering challenges encountered and resolved during development — particularly around API reliability and stateful conversation handling — reflect practical skills directly applicable to deploying LLM-based systems in real-world, latency- and reliability-sensitive environments.